DELBREIL Laura

Groupe 1

CO8SFPI0 - Rapport technique final

PROJET INFORMATIQUE INDIVIDUEL

“COONY”

Laurent MOULUQUET

20 avril 2026

SOMMAIRE

INTRODUCTION.....4

PRÉSENTATION DU PROJET & ANALYSE DU BESOIN.....5

> Problématiques identifiées : les barrières de la communication émotionnelle..... 5

> Solutions de soutien : 3 fonctionnalités clés..... 5

> Cadre éthique et engagement non médical..... 6

GESTION DE PROJET..... 7

> Organisation générale..... 7

> Plannings initial et prévisionnel - intermédiaire - final.....7

PHASES DE RECHERCHE ET DE CONCEPTION..... 10

> Définition du Minimum Viable Product (MVP).....10

> Maquettage..... 11

> Schéma de la base de données..... 12

CHOIX TECHNOLOGIQUES & ARCHITECTURE TECHNIQUE.....13

> Choix technologiques..... 13

> Organisation du code.....15

RÉALISATION TECHNIQUE ET POINTS COMPLEXES..... 17

> Fluidité des animations..... 17

> Service de recommandation..... 17

> Gestion des données.....18

> Interface et ergonomie..... 19

> Gestion globale de l'état..... 20

BILAN DU PROJET ET PERSPECTIVE D'AMÉLIORATIONS..... 21

> Objectifs atteints..... 21

> Difficultés rencontrées..... 21

> Pistes d'évolution.....22

BILAN PERSONNEL.....23

ANNEXES.....24

1. RECHERCHES X ÉTAT DE L'ART.....24
2. BENCHMARK..... 28
3. MVP : Minimal Viable Product..... 29
4. MAQUETTE FONCTIONNELLE FIGMA.....31
5. SCHÉMA DE LA BASE DE DONNÉES..... 32
6. ARCHITECTURE DE L'APPLICATION..... 33
7. DIAGRAMME DE GANTT : PLANNING RÉEL.....36
8. PROCESSUS D'INSTALLATION..... 37

INTRODUCTION

Le projet COONY s'inscrit dans le cadre du projet informatique individuel effectué lors du semestre 8. Il consiste en la conception et le développement d'une application mobile destinée à accompagner les enfants présentant un trouble du spectre de l'autisme (TSA) dans la compréhension, l'identification et l'expression de leurs émotions.

En effet, la reconnaissance et la verbalisation des émotions représentent un défi pour ces enfants, rendant parfois la communication complexe, tant pour eux que pour leur entourage. Aujourd'hui, les solutions de suivi sont souvent manuelles ou peu adaptées au quotidien. En tant que future ingénieure cogniticienne, il m'a semblé utile d'utiliser les compétences que j'ai acquises durant ma formation pour créer un outil de médiation émotionnelle accessible à ces enfants.

La solution consiste en une application composée de deux espaces complémentaires :

- **Espace enfant** : Une interface ludique et épurée permet d'exprimer son ressenti via un questionnaire guidé par des supports visuels (pictogrammes, couleurs). Ce parcours débouche sur des mini-jeux d'accompagnement émotionnel pour offrir un soutien immédiat.

- **Espace parent** : Un espace dédié permet de consulter le suivi émotionnel de l'enfant, offrant ainsi aux tuteurs une meilleure visibilité pour adapter leur accompagnement au quotidien.

Vous trouverez l'intégralité du code sur le lien suivant : **GIT COONY**

PRÉSENTATION DU PROJET & ANALYSE DU BESOIN

Dans un premier temps, il était essentiel de définir le cadre du projet ainsi que les besoins auxquels l'application devait répondre. Pour ce faire, une analyse des besoins utilisateurs a été réalisée. L'objectif était de mettre en évidence les difficultés rencontrées par les enfants porteurs de TSA dans l'identification et la communication de leurs émotions.

> Problématiques identifiées : les barrières de la communication émotionnelle

La conception de COONY se base sur des difficultés et répond à des problématiques clés, connues de la littérature scientifique sur le TSA :

- **Alexithymie / Régulation émotionnelle :** De nombreux enfants TSA présentent des difficultés de régulation émotionnelle, souvent corrélées à une forte anxiété. Des études suggèrent que ces difficultés sont exacerbées par l'alexithymie, définie comme l'incapacité à identifier et à décrire ses propres émotions. Selon l'étude *Investigating alexithymia in autism (2018)*, cette condition expliquerait une grande partie des difficultés sociales souvent attribuées au diagnostic du TSA.

- **Intéroception / Conscience du corps :** L'expression des émotions est fortement reliée à la perception des signaux corporels. Des recherches publiées dans *ScienceDirect (2022)* ont montré qu'un déficit d'intéroception impacte directement la conscience émotionnelle.
- **Connexion Parent et Enfant TSA :** Pour les parents et tuteurs, la difficulté à décoder les états émotionnels internes de l'enfant peut engendrer un fort sentiment d'impuissance et complexifier l'accompagnement au quotidien.

> Solutions de soutien : 3 fonctionnalités clés

COONY propose une solution structurée, inspirée de cadres cliniques reconnus tel que le modèle *SCERTS*, un cadre éducatif et clinique priorisant la Régulation Émotionnelle (ER) et le Soutien Transactionnel (ST).

- **Questionnaire émotionnel & design inclusif :** Dans l'interface, on remplace le vocabulaire complexe par des supports visuels et corporels. On s'inspire également de la méthode *The Incredible 5-Point Scale* pour quantifier l'intensité émotionnelle de 1 à 5, et du cadre *The Zones of Regulation* pour associer des couleurs aux états internes. Cette approche réduit la charge cognitive en proposant un nombre d'options limité par écran et une navigation stable, conformément aux recommandations de l'article *ACM* sur le *design orienté accessibilité TSA*.

- **Système de recommandation à base de règles :** Afin de garantir l'explicabilité et la sécurité de l'outil, l'application se base sur un système de règles transparentes, soit un mapping précis entre émotion, intensité, signal corporel et une stratégie

d’apaisement et de compréhension adaptée. Ce choix technique permet un contrôle total et une éthique de traitement des données.

- **Mini-jeux de soutien immédiat :** L’application intègre des mini-jeux “sérieux” comme premier niveau de soutien non médical. Ce concept s’appuie notamment sur les protocoles de la revue *Cochrane* concernant l'efficacité des technologies numériques pour la reconnaissance émotionnelle dans le TSA.

> Cadre éthique et engagement non médical

Étant donné la sensibilité des données traitées, l’application COONY applique les recommandations de la *CNIL* et du *Children's Code (ICO)* :

- **Minimisation des données** : collecte limitée au strict nécessaire et absence de capteurs intrusifs.
- **Stockage des données en “local”** grâce à une base de données SQLite.
- **Transparence** : Outil défini comme une aide à la communication et non un dispositif de diagnostic médical.

GESTION DE PROJET

> Organisation générale

Pour réaliser ce projet, j’ai défini plusieurs fonctionnalités et objectifs.

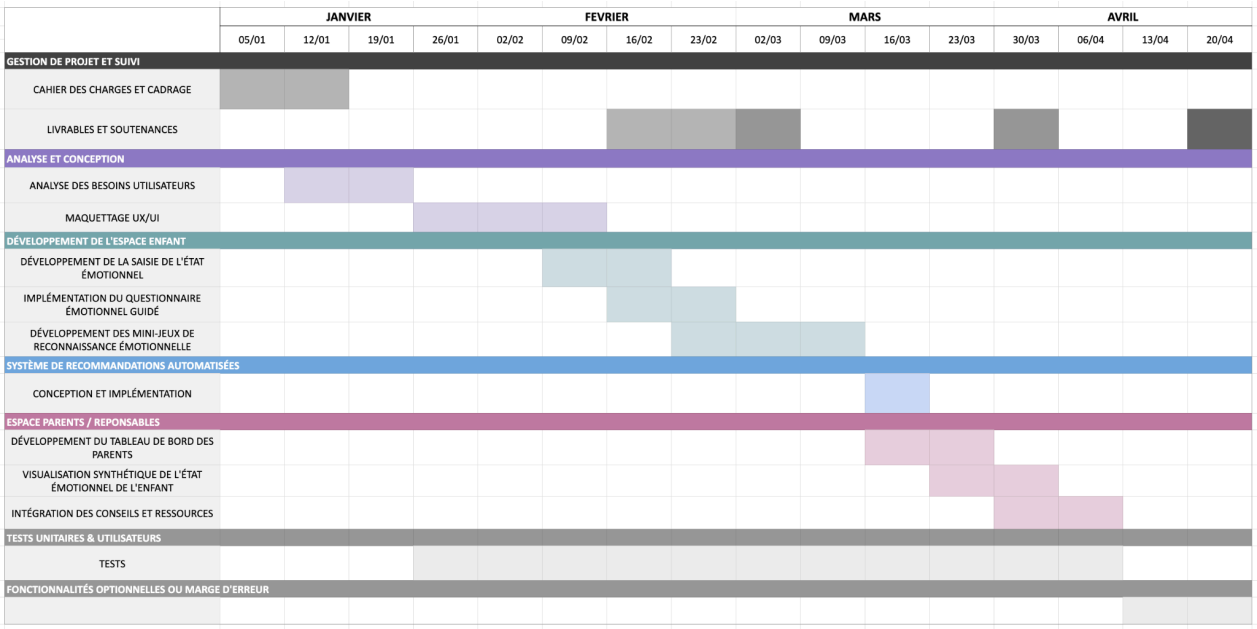
Dans un premier temps, je souhaitais que mon application permette aux enfants et aux parents de gérer l'intégralité du suivi émotionnel. Cela incluait, selon mes premières réflexions, le développement complet de l'espace enfant avec de nombreux mini-jeux de reconnaissance émotionnelle, un espace parents très fourni intégrant une visualisation synthétique complexe et l'intégration de conseils et ressources, ainsi qu’un questionnaire émotionnel lié à un système de recommandation. Cependant, lors de mon premier rendez-vous avec mon tuteur de projet, je me suis rendu compte que mon projet était certainement trop ambitieux pour le temps qui m’était imparti. J’ai donc décidé, selon les conseils de mon tuteur, d’établir le MVP de ce projet et de réduire le nombre de fonctionnalités pour me concentrer sur le "cœur" de l'application : le parcours émotionnel guidé et la régulation. (cf. PHASES DE RECHERCHE ET DE CONCEPTION)

J’ai ensuite créé un planning prévisionnel reprenant ces différentes fonctionnalités réparties en plusieurs objectifs de conception et de développement.

> Plannings initial et prévisionnel - intermédiaire - final

- **V1** : PLANNING PRÉVISIONNEL INITIAL

Conçue en autonomie, cette première version sous-estimait le temps de recherche et la complexité de l'environnement technique. Elle prévoyait un développement frontal de tous les espaces simultanément dès février, ce qui aurait dispersé les efforts.



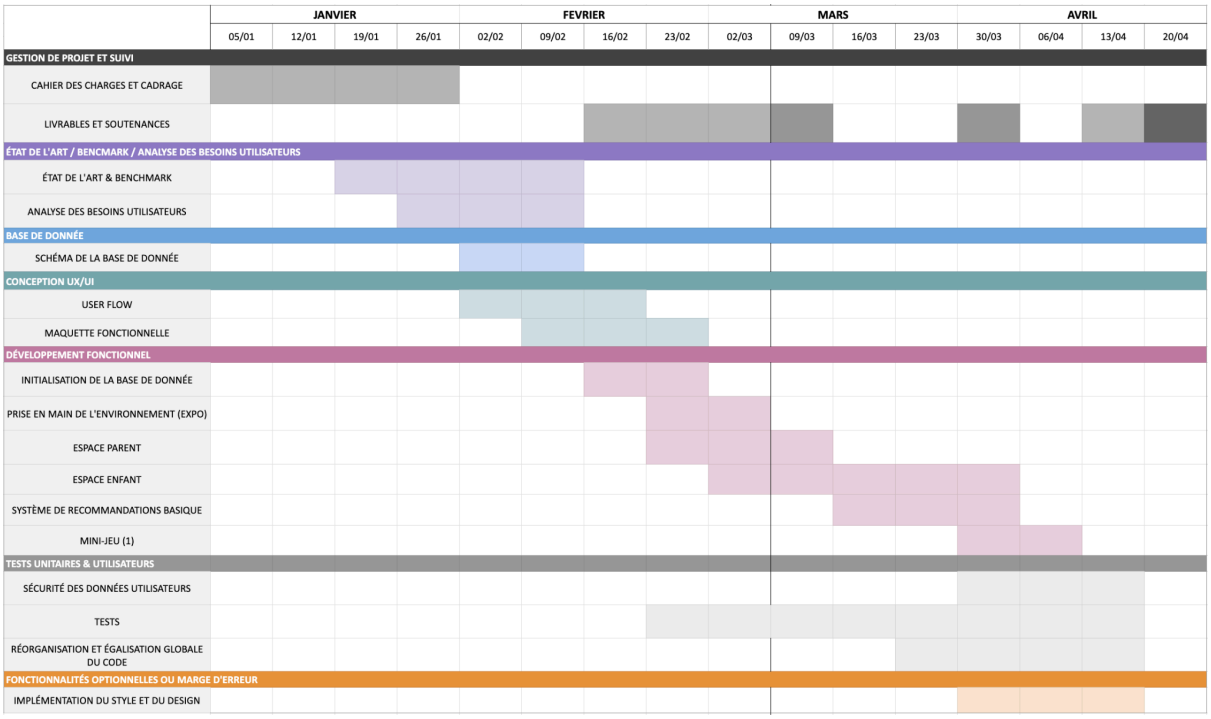
- **V2** : PLANNING PRÉVISIONNEL REVU

Revu en collaboration avec le tuteur, ce planning a permis de hiérarchiser les priorités. Nous avons décalé le développement fonctionnel pour laisser place à une phase de conception UX/UI plus approfondie, indispensable pour un public TSA.

- **V3** : POINT INTERMÉDIAIRE

Donné dans le rapport intermédiaire, ce planning montre le début de la phase de développement fonctionnel. À ce stade, l'ensemble des recherches était finalisé, la maquette fonctionnelle réalisée et le socle de la base de données stabilisé, permettant de lancer l'implémentation des différents espaces Enfant et Parent.

- **V4** : PLANNING RÉEL FINAL (*cf. Annexe 7*)



Le planning final montre une répartition plus dense et réaliste du travail sur les deux derniers mois :

- **Espace Enfant (mars - avril)** : Cette phase a été plus longue que prévu. Ce délai s'explique par la complexité de la connexion entre cet espace et l'ensemble des autres composants déjà implémentés. De plus, cette partie centrale englobe le questionnaire qui est l'une des parties clés de l'application.

- **Le Système de Recommandation (mars)** : Implémenté simultanément avec l'espace enfant, il a servi de pont logique entre les données de l'enfant et les mini-jeux proposés. Une fois les deux parties individuellement créées, il a fallu un temps pour assembler et faire fonctionner ces parties ensemble.

- **Réorganisation globale du code (avril)** : Une étape cruciale et non négligeable qui a consisté à nettoyer et réorganiser le code après que l'ensemble des parties a été développé. Cela a permis de garantir un projet maintenable et professionnel.

- **Implémentation du style et du design (avril)** : Étape non incluse dans le planning, cette phase a été menée dans la période initialement prévue pour l'ajout de fonctionnalités supplémentaires. Malgré l'échéance du projet, j'ai choisi d'investir ce temps pour intégrer la direction artistique finale (animations, fond mouvant, charte graphique), faisant passer le projet d'un prototype fonctionnel à une application plus complète et immersive.

PHASES DE RECHERCHE ET DE CONCEPTION

Dans la continuité de l'analyse, une étude de benchmark et un état de l'art scientifique ont été réalisés afin d'identifier les approches existantes, ou non, les plus pertinentes pour accompagner l'expression émotionnelle chez les enfants présentant un TSA. De plus, ces recherches ont permis de définir certains principes essentiels à prendre en compte dans la création de l'application, notamment l'utilisation de supports visuels simples, une interaction guidée et un accompagnement de l'enfant tout au long de son expérience.

(cf. ANNEXE 1 & 2)

> Définition du Minimum Viable Product (MVP)

(cf. ANNEXE 3)

Le projet COONY étant ambitieux, il a été nécessaire d'établir au plus tôt les limites de ce que nous souhaitions réaliser durant la période qui nous était allouée. Pour cela, nous avons établi le MVP qui se compose de :

- **Questionnaire émotionnel guidé** : Point central de l'application, ce module permet à l'enfant de compléter un questionnaire simplifié et dont la charge mentale est délibérément allégée grâce à :
    - Sélection d'émotions associées à des couleurs et pictogrammes.
    - Choix d'intensité sur une échelle de 1 à 5 visuelle et défini par des mots simples.
    - Identification optionnelle du ressenti corporel, afin de favoriser l'interoception, grâce à des mots simples et des pictogrammes.
- **Visualisation et validation des réponses :** Après chaque question et questionnaire, l'interface propose un feedback visuel après chaque saisie afin de rassurer l'enfant et de l'aider à valider son choix grâce à un message bienveillant. On transforme ainsi l'expérience en une forme d'échange positif.
- **Système de recommandation algorithmique :** En fonction des réponses au questionnaire, l'application propose automatiquement un mini-jeu de soutien émotionnel adapté grâce à un système de règles simples.
- **Mini-Jeux :** Le MVP intègre a minima un mini-jeu complet et fonctionnel. Ici, le choix s'est porté sur un mini-jeu de respiration et d'apaisement.
- **Connexion - Inscription - Espace parent :** Le projet inclut un système fonctionnel de création de compte et de connexion pour le parent ainsi que l'accès à un compte et à

un tableau de bord simplifié, permettant au parent de visualiser l'historique émotionnel de l'enfant ainsi que son état le plus récent enregistré.

Cette version initiale se base sur les principales problématiques et solutions choisies afin de créer une solution fonctionnelle et facilement améliorable. Les évolutions futures pourraient inclure des personnalisations des préférences sensorielles ou encore un tableau de bord plus avancé, incluant des statistiques et des tendances par exemple.

> Maquettage

Pour poser les premières bases de l'application, une maquette fonctionnelle a été réalisée sur papier puis sur Figma. L'objectif était de poser les premières idées sur la base des recherches qui avaient été réalisées afin de définir les différents écrans de l'application ainsi que le parcours utilisateur.

(cf. ANNEXE 4)

Cette maquette servant de base de travail, les choix fonctionnels et techniques ont été privilégiés par rapport au design et à l'aspect esthétique qui seront ajoutés plus tardivement. Pour un MVP, nous avons privilégié la fonctionnalité de l'application avant tout.

Nous nous sommes basés sur les recherches effectuées précédemment pour délimiter quelques points clés qu'il était pour nous essentiel d'intégrer dans la maquette tel que :

- **Réduction de la charge cognitive** : Limiter les écrans à une seule action / question afin d'éviter la surcharge d'informations, notamment pour la partie questionnaire.
- **Stabilité et prévisibilité de la navigation** : Proposer une navigation constante et des éléments interactifs ayant des emplacements définis tout au long du parcours utilisateur, afin de créer une routine numérique sécurisante.
- **Tolérance à l'erreur** : L'interface propose des sorties facilitées et des retours en arrière constants ainsi que des confirmations pour des actions critiques, évitant ainsi les frustrations et les sentiments de blocage éventuel.
- **Affordance des éléments** : Insérer des boutons assez larges, arrondis et contrastés pour faciliter les manipulations.

Pour la partie du questionnaire émotionnel, il était pour nous important de segmenter et de fluidifier les différentes étapes telles que :

- Identification de l'**émotion** parmi une sélection simplifiée.
- **Graduation** de l'intensité.

- Localisation de **signaux corporels** simples.
- Récupération du **lieu** de réponse pour favoriser la connexion avec l'environnement.
- **Validation et feedback** des réponses pour favoriser le ressenti de l'enfant et son accompagnement.

Une fois les fonctionnalités principales identifiées et maquettées, un prototype fonctionnel a vu le jour, servant de base pour commencer le développement de l'application.

> Schéma de la base de données

Tout au long de la conception de la maquette fonctionnelle, un schéma de base de données a été conçu dans l'objectif de structurer les différentes informations utilisées dans l'application. Cette base de données comporte plusieurs tables principales permettant de stocker les données fonctionnelles de l'application : comptes parents, profils enfants, questionnaires émotionnels, catalogues d'émotions, etc. Les tables catalogues permettent de faciliter l'ajout de nouveaux mini-jeux par exemple, ne nécessitant pas de refonte globale mais une simple incrémentation de la table catalogue.

(cf. ANNEXE 5)

CHOIX TECHNOLOGIQUES & ARCHITECTURE TECHNIQUE

Globalement, l'architecture de COONY a été pensée pour offrir une expérience fluide sur mobile tout en garantissant une protection maximale des données de l'enfant.

> Choix technologiques

- **Environnement technique : React Native & Expo**

Pour l'environnement technique, le choix s'est porté sur l'écosystème React Native couplé à Expo pour plusieurs raisons stratégiques. Cette application permet de maintenir une base de code unique pour iOS et Android, assurant une cohérence visuelle et fonctionnelle. De plus, l'utilisation d'Expo accélère les cycles de test et de déploiement, facilitant l'ajustement de l'interface en temps réel. Finalement, React Native permet d'intégrer des animations fluides essentielles pour les mini-jeux de régulation, tout en conservant la légèreté d'une application mobile moderne.

- **Gestion de données : SQLite**

Il avait initialement été annoncé que Firebase serait utilisé pour la gestion de la base de données. Afin de faciliter cette étape, et étant encore peu familière avec la technologie Firebase, j'ai commencé par utiliser SQLite, que je connaissais déjà, pour implémenter une première version de la base de données avant d'envisager une migration vers Firebase. Je me suis finalement rendu compte que la version SQLite était largement suffisante pour les besoins du projet et j'ai donc décidé de conserver cette version déjà fonctionnelle afin de gagner du temps dans le développement.

Stocker localement les données permet de sécuriser les informations ainsi que de limiter les risques liés aux serveurs distants. Finalement, une liaison fonctionnelle a été établie entre l'application et la base de données, notamment grâce à l'implémentation de requêtes métier fonctionnelles.

Au fur et à mesure de l'avancement du projet, certaines modifications ont été apportées à cette base de données. La table catalogue couleurs a par exemple été supprimée et remplacée par un fichier directement dans le code. Il semblait finalement plus logique de garder cette partie dans la partie code, notamment pour des questions de facilité de modification.

- **Gestion des états : context API & expo-router**

La navigation et la logique d'état global de l'application sont orchestrées pour simplifier le parcours parent et enfant. Context API a été utilisé pour maintenir l'état global de l'application, tel que l'identité du parent connecté et le profil de l'enfant sélectionné, évitant de recharger les informations à chaque changement d'écran. Expo-Router permet une

gestion de la navigation basée sur les fichiers, offrant une transition fluide entre le questionnaire et les mini-jeux. Enfin, cette architecture garantit que les recommandations de mini-jeux sont injectées dynamiquement en fonction des réponses stockées temporairement dans l'état global avant leur persistance en base de données.

- **Langage et structure de code : JavaScript (JS & JSX)**

Le développement de COONY repose intégralement sur l'écosystème JavaScript, choisi pour sa flexibilité et sa vaste communauté de support. D'un côté, JavaScript a été utilisé pour l'ensemble de la logique métier de l'application, et de l'autre JSX, a été utilisé pour développer l'ensemble des structures d'interface directement au sein du code JavaScript. Ce choix permet de lier la logique de l'application aux composants visuels. Enfin, la séparation entre les fichiers .js de logique et .jsx de composants d'interface permet une maintenance simplifiée et une meilleure lisibilité du code, facilitant les évolutions futures du projet.

> Organisation du code

L'arborescence du projet a été structurée de façon modulaire, en isolant la logique métier de l'interface utilisateur. Cette organisation permet une meilleure lisibilité et une gestion efficace des données traitées localement.

(cf. ANNEXE 6)

- **Présentation des dossiers et modularité globale**

Le projet s'organise autour de plusieurs répertoires centraux, chacun ayant un rôle clé dans l'architecture de l'application.

- **/app** : Ce dossier contient le cœur de la navigation de l'application via expo-router. Il se base sur un système de routage par fichier, segmenté par types d'utilisateurs et d'utilisation : /auth, /enfant et /parent. Il contient également les fichiers de structure globaux (*_layout.js*).
- **/screens** : Ce dossier contient les différentes pages de l'application (fichiers format .jsx) telles que *PageTableauBordParent.jsx* ou encore *IntroQuestionnaire.jsx*. Cette partie fait le lien entre les composants et la logique de navigation.
- **/src** : Ce répertoire contient l'ensemble de la logique métier de l'application
    - **/src/components** : Ce sous-dossier regroupe l'ensemble des éléments d'interfaces et réutilisables. Ils sont organisés par thématiques :
        - **common/** : Boutons, en-têtes génériques ..
        - **Mini_Jeux/** : Composants spécifiques aux mini-jeux de l'application, comme *Mini_Jeu_Respiration.jsx* par exemple.
        - **dashboard/** et **parent/** : éléments dédiés aux vues de suivi et de gestion.
    - **/src/data** : Dossier crucial qui s'occupe de l'interaction avec la base de données SQLite, il contient :
        - **sqlite/** : Contient la configuration du client, des requêtes SQL brutes (*queries.js*) et le schéma de la base de données (*schema.js*).
        - **repositories/** : Intègre le "Repository Pattern", tel que *enfant.repo.js*, permettant d'isoler les appels à la base de données du reste de l'application.

- **Gestion des états et logique métier**

Pour assurer la fluidité de l'expérience utilisateur et la persistance des choix, l'application s'appuie sur des répertoires spécialisés :

- **/src** : Ce fichier contient l'ensemble de la logique métier de l'application
    - **/src/state** : Gère les états globaux de l'application via la Context API, pour suivre l'enfant sélectionné par exemple (*enfantSelectionne.js*) ou encore l'état du questionnaire en cours.
    - **/src/hooks** : Contient les hooks qui englobent la logique complexe, tel que *useParentTableauDeBord.js*, permettant d'alléger les composants visuels en distinguant la récupération et le traitement des données.
    - **/src/utils :** Regroupe les fonctions utilitaires transversales :
        - **mapper/** : Permet de transformer les données brutes de la base en objets utilisables par les composants (*emotionMapper.js*).
        - Outils de validation, gestion des dates et de sécurité (*hash.js*).

- **Langages et convention de nommage**

Le projet utilise une distinction claire entre la logique et le rendu, les fichiers .js sont privilégiés pour la logique pure (repositories, services, etc.), tandis les fichiers .jsx sont utilisés pour les composants React Native, ce qui facilite l'identification des fichiers dans la structure globale de l'interface.

RÉALISATION TECHNIQUE ET POINTS COMPLEXES

> Fluidité des animations : src/components/Mini_Jeux/Mini_Jeu_Respiration.jsx

L'un des points complexes d'implémentation de l'architecture front-end est la gestion des animations visuelles fluides. Utile dans l'implémentation des mini-jeux par exemple, ou encore pour l'ajout des vagues dans l'arrière-plan de l'application. L'objectif est de rendre l'application la plus fluide et agréable possible pour des enfants.

Si on regarde le mini-jeu de respiration guidée, l'Animated API de React Native a été utilisée, permettant de définir une trajectoire d'animation complète au lieu d'utiliser un simple changement d'état qui forcerait l'application à recalculer le rendu à chaque milliseconde.

```
setEtape("J'inspire...");
Animated.timing(scaleAnim, {
  toValue: 2,
  duration: 4000,
  easing: Easing.inOut(Easing.sin),
  useNativeDriver: true,
}).start(() => {

  setEtape("Je bloque un instant...");
  setTimeout(() => {

    setEtape("J'expire doucement...");
    Animated.timing(scaleAnim, {
      toValue: 1,
      duration: 4000,
      easing: Easing.inOut(Easing.sin),
      useNativeDriver: true,
    }).start(() => {
      setCyclesTermines(prev => prev + 1);
    });

  }, 2000);
});
```

En ajoutant *useNativeDriver* : true, l'animation est déportée du thread JavaScript (souvent occupé par la logique métier) vers le thread natif du téléphone. Cela garantit une animation fluide même si l'application effectue des opérations plus lourdes en parallèle.

Dans ce code, la variable *scaleAnim* contrôle la propriété *transform: [{scale}]* de la bulle. Grâce à *(Easing.sin),* on utilise une courbe d'accélération sinusoïdale, permettant de mimer le rythme naturel de la respiration.

Cette fluidité est d'autant plus importante qu'elle aide l'enfant à visualiser et donc se synchroniser avec le mouvement fluide de la bulle.

> Service de recommandation : src/services/recommendation.service.js

Le cœur fonctionnel de l'application repose sur le système de recommandation de mini-jeux à partir des réponses du questionnaire émotionnel. Cette recommandation s'effectue via un algorithme de sélection d'activité qui analyse simultanément plusieurs variables.

```
export const REGLES_RECOMMANDATIONS = [
  {
    // Colère forte
    id_emotion: 4,
    intensites_valides: [4, 5],
    signaux_valides: [1, 4],
    reponse: {
      titre: "Tu te sens en colère.",
      message: "C'est une émotion qui peut être forte.",
      proposition: "On va respirer doucement ensemble pour calmer ton corps.",
      idMiniJeu: 1, // Respiration 4-4
    }
  },
```

L'enjeu de ce choix technique consistait à créer un système permettant de prendre en compte non seulement l'émotion choisie

(question 1) mais aussi l'intensité donnée (question 2) et les signaux physiques ressentis par l'enfant (question 3). Pour y parvenir, un service de "matching" a été implémenté, permettant de comparer les données du questionnaire à une matrice de règles prédéfinies.

```
export function matchRecommandation(idEmotion, intensite, idSignal) {
  const match = REGLES_RECOMMANDATIONS.find(regle => {
    return (
      regle.id_emotion === idEmotion &&
      regle.intensites_valides.includes(intensite) &&
      regle.signaux_valides.includes(idSignal)
    );
  });

  if (match) return match.reponse;

  const matchPartiel = REGLES_RECOMMANDATIONS.find(regle => {
    return (
      regle.id_emotion === idEmotion &&
      regle.intensites_valides.includes(intensite)
    );
  });

  if (matchPartiel) return matchPartiel.reponse;
```

Ici, on voit que l'algorithme utilise une stratégie de "fallback", il cherche d'abord la règle la plus précise pour offrir la recommandation la plus adaptée. Si aucun signal physique spécifique ne correspond, il redescend vers une règle basée sur les émotions et l'intensité associée données, l'objectif est de ne jamais laisser l'enfant sans réponse.

> Gestion des données : src/data/sqlite/queries.js

Bien que plus simple à mettre en œuvre, l'utilisation de SQLite permet également de garantir la sécurité des informations sensibles liées à l'enfant. Le stockage se fait directement dans la mémoire locale de l'appareil, garantissant que les données ne transitent pas sur un serveur.

Pour utiliser cette technologie, il fallait réussir à passer d'une structure de données NoSQL, (ici un format JSON simple), à une structure relationnelle SQL. Plusieurs tables ont été implémentées, telles que questionnaire_emotionnel et catalogue_emotions, liées par des clés étrangères pour garantir l'intégrité des données.

Pour assurer la liaison entre les différents espaces, pour que le parent puisse consulter les informations provenant du questionnaire de son enfant, le code doit récupérer et reconstruire l'information en croisant les tables. Pour cela, des requêtes utilisant des LEFT JOIN ont été implémentées.

```
export const selectQuestionnairesByEnfant = (idEnfant) =>
  queryMany(
    `SELECT
      q.*,
      e.libelle as emotion_nom,
      l.libelle AS lieu_nom,
      s.libelle AS signal_nom
     FROM questionnaire_emotionnel q
     LEFT JOIN catalogue_emotions e ON q.id_emotion = e.id_emotion
     LEFT JOIN catalogue_lieux l ON q.id_lieu = l.id_lieu
     LEFT JOIN catalogue_signaux_corporels s ON q.id_signal_corporel = s.id_signal_corporel
     WHERE q.id_enfant = ?
     ORDER BY q.date_questionnaire DESC`,
    [idEnfant]
  );
```

Par exemple, cette requête est centrale, elle permet d'extraire de la table *questionnaire_emotionnel* les données brutes, tout en allant chercher les libellés des émotions, des lieux et des signaux corporels dans leurs catalogues respectifs.

> Interface et ergonomie : src/utils/mapper/emotionMapper.js

L'objectif était de transformer une donnée brute, ici l'ID d'une émotion, en une expérience qualitative immédiate. Plutôt que de gérer les styles au cas par cas, l'identité visuelle de l'application a été centralisée dans des objets de référence. Notamment pour les informations relatives aux émotions. Cela permet d'assurer une cohérence absolue, ainsi, la colère sera toujours représentée par le rouge et l'icône "angry", quel que soit l'écran (questionnaire ou historique parent).

```
export const EMOTIONS = {
  1: {
    id: 1,
    label: "Joyeux",
    color: "#FFD700",
    iconName: "emoticon-outline",
  },
  2: {
    id: 2,
    label: "Triste",
    color: "#42A5F5",
    iconName: "emoticon-sad",
  },
  3: {
    id: 3,
    label: "Calme",
    color: "#66BB6A",
    iconName: "emoticon-happy",
  },
```

L'utilisation de la fonction *mapEmotion* permet de lier dynamiquement le style des composants à l'état de l'application. L'enfant ne lit pas seulement un mot, il "voit" l'émotion à travers le code couleur.

La fonction prévoit également un cas de repli, le label "Inconnu" avec une couleur neutre. Cela évite un plantage de l'interface si une donnée est mal formatée, garantissant une stabilité pour l'utilisateur.

En associant une icône (*iconName*) à chaque émotion, on permet une navigation par pictogrammes, très utile

pour les enfants ayant des difficultés à exprimer et comprendre leurs émotions.

> Gestion globale de l'état via Context API dans le questionnaire

Le questionnaire émotionnel de l'enfant est fragmenté en quatre étapes (Émotion, Intensité, Signal corporel, Lieu). Utiliser un état local à chaque page aurait rendu la sauvegarde finale impossible. J'ai donc mis en place un *QuestionnaireProvider* utilisant l'API Context de React. Ce mécanisme permet de créer un "magasin de données" global qui enveloppe l'ensemble de l'application.

```
const QuestionnaireContext = createContext();

export function QuestionnaireProvider({ children }) {
  const [questionnaire, setQuestionnaire] = useState({
    idEnfant: null,
    prenom: null,

    idEmotion: null,
    emotionLabel: null,
    intensiteEmotion: null,

    idSignalCorporel: null,
    signalLabel: null,

    idLieu: null,
    lieuLabel: null,
  });

  const mettreAJourQuestionnaire = (nouvellesValeurs) => {
    setQuestionnaire((precedent) => ({
      ...precedent,
      ...nouvellesValeurs,
    }));
  };
```

Grâce à la fonction *mettreAJourQuestionnaire* et à l'opérateur de décomposition (*...precedent*), chaque écran de question vient enrichir l'objet global. Par exemple, l'écran avec la question "Lieu" ajoute son information sans supprimer l'émotion choisie trois écrans plus tôt. Les composants d'interface n'ont pas besoin de savoir comment la donnée est stockée. Ils utilisent simplement le hook personnalisé *useQuestionnaire()* pour envoyer leur mise à jour. Cette structure garantit que la fonction d'insertion en base de données SQLite dispose toujours d'un objet complet et valide au moment de la validation finale par l'enfant.

BILAN DU PROJET ET PERSPECTIVE D'AMÉLIORATIONS

> OBJECTIFS ATTEINTS

En cette fin de projet, nous pouvons dire que l'application COONY a dépassé le stade du simple prototype technique, c'est une application complète intégrant aussi bien un aspect fonctionnel qu'esthétique abouti. La solution finale correspond aux attentes fonctionnelles du MVP enrichi par un design global adéquat très complet.

Outre les fonctionnalités et technologies expliquées précédemment, l'implémentation de la charte graphique participe aussi bien à l'aspect esthétique qu'à l'utilisabilité pour les enfants TSA, créant un environnement numérique stable, rassurant et apaisant.

Le système du questionnaire émotionnel guidé associé à la recommandation de mini-jeux, couplé à la base de données SQLite permet une expérience fluide et adaptée aux plus jeunes.

> DIFFICULTÉS RENCONTRÉES

Le développement en lui même a présenté quelques défis techniques et personnels :

Ayant une seule année d'expérience en programmation à mon actif, j'ai dû faire preuve d'autonomie et de patience pour apprendre à maîtriser l'écosystème Expo et développer mes compétences en React Native. Pour pallier mes lacunes et respecter les délais, j'ai combiné recherche documentaire, utilisation stratégique de LLM pour comprendre des concepts complexes, mais j'ai aussi sollicité des amis ayant des compétences bien plus avancées. Ces échanges ont été essentiels pour débloquer certains points techniques critiques et m'ont permis de progresser bien plus rapidement que par un apprentissage isolé.

L'architecture du projet n'a pas été figée dès le départ ; elle s'est construite de manière itérative. Cette approche a parfois engendré des phases de restructuration complète. Bien que chronophages, ces étapes ont été très enrichissantes, puisqu'elles m'ont forcée à repenser la modularité de mes composants et de ma base de données pour rendre le code aussi clair et maintenable que possible.

Initialement, le MVP devait se concentrer sur une application strictement fonctionnelle. Cependant, j'ai décidé de me lancer le défi d'implémenter une véritable direction artistique avec un design soigné et des fonctionnalités visuelles avancées, comme le fond mouvant. Mon objectif était que l'application soit la plus proche possible de la vision que j'en avais. L'intégration de ces éléments esthétiques, couplée à l'apparition de nouvelles idées au fil de

l'implémentation, a représenté une charge de travail importante et a nécessité un investissement temporel conséquent.

> PISTES D'ÉVOLUTION

Bien que le projet soit bien avancé et le MVP respecté, certaines fonctionnalités peuvent être améliorées, enrichies et ajoutées à l'application actuelle :

- **Amélioration fonctionnelles et pédagogiques :**
    - **Extension de la bibliothèque de mini-jeux** : l'une des priorités est d'implémenter l'ensemble des autres mini-jeux pour offrir réellement une réponse spécifique à chaque combinaison du questionnaire.
    - **Page de conseils pour les parents** : L'un des objectifs de l'application étant d'aider les parents à comprendre les émotions de leurs enfants, il est nécessaire d'ajouter une page conseils personnalisés et adaptée aux besoins de l'enfant et des parents. Le bouton étant déjà implémenté sur la page *TableauBordParent.jsx*, il suffira d'ajouter simplement la page *Conseil.jsx*.
    - **Catalogue des émotions** : Comme l'application est dédiée à la compréhension des émotions, il semble logique d'ajouter une partie dédiée à la présentation et à l'explication de celles-ci. Le bouton étant déjà présent dans *TableauBordEnfant.jsx*, il suffira d'ajouter la ou les pages associées à cette nouvelle fonctionnalité.
    - **Scinder l'application en deux applications complémentaires** : avec une application parent qui donnera accès uniquement aux fonctionnalités liées aux parents et une application enfant qui donnera respectivement accès uniquement au dashboard enfant connecté.
- **Accessibilité et Stockage des données :**
    - **Synthèse vocale :** Intégrer la lecture audio et la description audio des questions et des mini-jeux pour rendre l'application accessible aux enfants ne sachant pas lire.
    - **Fichier csv. comme base de recommandation** : L'utilisation d'un fichier CSV permettrait d'externaliser la logique de recommandation pour la rendre plus facile à modifier sans toucher au code. Cette approche offre une meilleure flexibilité et simplifie l'évolution future de l'application.
    - **Migration Cloud** : Pour permettre une synchronisation entre plusieurs appareils et augmenter la portée de l'application, il serait nécessaire de passer sur une architecture de type Firebase par exemple.

BILAN PERSONNEL

Ce projet m’a permis d’acquérir de nombreuses compétences dans le développement mobile et la conception inclusive. Tout d’abord, j’ai pu apprendre à utiliser et développer ma maîtrise des technologies comme React Native et l'écosystème Expo. J'ai également approfondi mes connaissances en gestion de données locales avec SQLite, un choix technique crucial pour garantir la confidentialité des données sensibles des utilisateurs.

J’ai également développé des compétences solides en gestion de projet, notamment dans l’organisation modulaire du code, la planification des sprints pour atteindre le MVP, l’application de méthodes itératives et le respect des délais. De plus, ce projet a été l'opportunité idéale pour appliquer concrètement les connaissances en UX Design vues en cours, en les adaptant aux besoins très spécifiques des enfants présentant un trouble du spectre de l'autisme (sobriété visuelle, affordance, réduction de la charge cognitive).

Enfin, ce projet a été plus que bénéfique et enrichissant car il m'a permis de mettre mon apprentissage au service d'une cause qui me tient à cœur : l'inclusion et le soutien aux personnes ayant des besoins spécifiques. Créer COONY m'a donné la possibilité de transformer une réflexion théorique sur le handicap en une application fonctionnelle, concrète et utilisable, ce qui représente une grande source de satisfaction personnelle et professionnelle.

ANNEXES

1. RECHERCHES X ÉTAT DE L'ART

RECHERCHES

---

1) Problématique scientifique et concepts clés (TSA x émotions)

1.1. Difficultés émotionnelles dans le TSA

- **Emotion regulation (ER)** : de nombreux enfants/ados TSA présentent des difficultés de régulation émotionnelle, associées à anxiété, comportements défis, surcharge, etc.
  Emotion regulation.pdf

- **Alexithymie** (difficulté à identifier/décrire ses émotions) : hypothèse "alexithymia-driven" — une partie des difficultés d'émotions dans le TSA pourrait être mieux expliquée par l'alexithymie que par le TSA lui-même.
  investigating-alexithymia-in-autism-a-systematic-review-and-meta-analysis.pdf

- **Interoception** (perception des signaux corporels) : axe très lié à la conscience émotionnelle et à l'ER dans le TSA.
  https://www.sciencedirect.com/science/article/pii/S0891422222002086?utm_source=chatgpt.com

> **Design** privilégier des supports **visuels + corporels** (couleurs, intensité, signaux corporels simples), et des questions très courtes pour "mettre en mots" sans exiger un vocabulaire émotionnel riche.

2) Interventions "evidence-based" utiles comme socle (et quoi reprendre dans une app)

2.1. Rapports de référence "Evidence-Based Practices" (EBP)

- **NCAEP 2020** (UNC) : synthèse majeure des pratiques fondées sur les preuves pour enfants/ados/jeunes adultes autistes. EBP-Report-2020.pdf

  => Base pour justifier les choix de fonctionnalités (supports visuels, interventions médiées par les parents, compétences sociales, etc.).

2.2. Cadres et programmes souvent utilisés autour de l'ER

- **The SCERTS Model** : cadre centré sur *Social Communication*, *Emotional Regulation*, *Transactional Support* : modèle d'éducation innovant pour aider à travailler avec des enfants TSA et leurs familles : The-SCERTS-Model-for-Web-12-12-07.pdf

- **The Incredible 5-Point Scale** : outil visuel "échelle 1–5" pour rendre concret l'intensité, les attentes, les stratégies : The-incredible-5-point-scale.pdf

- **The Zones of Regulation** : cadre très diffusé (zones/couleurs + stratégies). Mais certaines analyses soulignent peu de preuves directes spécifiquement chez les personnes autistes, même si le cadre est "practice-based" et très utilisé. https://zonesofregulation.com/?utm_source=chatgpt.com

=> **Utile pour revendiquer une inspiration** (langage commun, échelles, stratégies) tout en restant prudente : *"inspiré de cadres de régulation émotionnelle largement utilisés, et aligné avec des pratiques EBP"*

3) Technologies numériques pour TSA

3.1. Apps/jeux pour reconnaissance émotionnelle et socio-émotionnel

- **Récap** sur les apps mobiles de reconnaissance des émotions pour enfants TSA : cartographie des outils existants + besoins non couverts : frcha-02-1118665.pdf

- **Jeux sérieux & neurodéveloppement** : https://www.sciencedirect.com/science/article/pii/S0149763423001252?utm_source=chatgpt.com

- **Cochrane protocol** sur jeux sérieux et reconnaissance des émotions dans l'autisme : CD014673.pdf

- **Exemple d'étude app-based (pilot RCT)** : intervention app (Yface) testée chez des enfants TSA avec objectifs de compétences sociales/perception faciale/regard : fpsyt-14-1126290.pdf

3.2. Reconnaissance d'émotions via capteurs / IA

- Systèmes de reconnaissance d'émotions et ML/capteurs en autisme : https://www.sciencedirect.com/science/article/pii/S1386505624001321?utm_source=chatgpt.com

=> Dans ce projet : plutôt se concentrer sur **auto-rapport guidé + mini-jeux** et éviter l'utilisation de l'IA et de capteurs par exemple.

4) Recommandations automatisées

- **Système à règles (rule-based)** : mapping {émotion + intensité + contexte} > {stratégies}.
    - Avantages : explicable, contrôlable, rassurant pour les parents/éthique.
- **Personnalisation légère** : préférences parent/enfant (activités aimées/évitées) + historique simple.
- **Recommandation "data-driven"** : nécessite beaucoup de données + enjeux RGPD/consentement + biais.

=> Dans ce rojet le **rule-based** est le meilleur compromis (explicabilité + sécurité).

5) UX / accessibilité pour enfants TSA

- Article ACM sur design orienté accessibilité avec focus autisme (requirements + design) : 020094_1_online.pdf

- Étude 2025 sur méthodes d'analyse des besoins UI pour apps d'intervention TSA (approche HCI) : fpsyt-16-1508006.pdf

=> GLOBALEMENT dans le cadre du projet COONY

- **Réduire la charge cognitive** : peu d'options par écran, navigation stable, icônes consistantes.
- **Support visuel** : pictos, couleurs, échelles d'intensité, feedback immédiat non punitif.
- **Prévisibilité** : mêmes emplacements, transitions douces, pas de surprises.
- **Tolérance à l'erreur** : annuler/retour facile, confirmation avant quitter.
- **Personnalisation** : niveau de complexité, choix d'émotions proposées, préférences sensorielles.

6) Éthique, RGPD, et cadre "non médical"

6.1. Données d'enfants : exigences renforcées

- Recommandations **CNIL** pour applications mobiles (privacy by design, minimisation, transparence, permissions) : https://www.cnil.fr/en/mobile-applications-cnil-publishes-its-recommendations-better-privacy-protection?utm_source=chatgpt.com

- "Children's code" / guidance risque-impact pour services numériques destinés aux enfants (DPIA, risques développementaux) : https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/childrens-code-guidance-and-resources/dpia-tools/mobile-gaming-app/step-5-identify-and-assess-risks/?utm_source=chatgpt.com

- Doc de référence sur approche "child-oriented" RGPD (mesures pratiques, protection par défaut). Fundamentals for a Child-Oriented Approach to Data Processing_FINAL_EN.pdf

6.2. Digital mental health chez les jeunes : bénéfices et risques

- Review sur enjeux éthiques des interventions numériques en santé mentale chez les jeunes (promesses, défis : confidentialité, accès, responsabilité, etc.) : fdgth-03-697072.pdf

- Revue sur l'usage d'apps/technos en santé mentale enfant/ado (détection/gestion/maintenance) : ebmental-2019-300093.pdf

=> GLOBALEMENT dans le cadre du projet COONY

- **Outil d'accompagnement ≠ diagnostic**

- **Minimiser les données** : pas de capteurs intrusifs par défaut, pas de collecte inutile, stockage sécurisé, contrôle parental + respect de l'intimité de l'enfant
- **Transparence** : expliquer clairement *quoi est stocké*, *pourquoi*, *combien de temps*, *qui peut voir quoi*.

7) Mesures d'évaluation

7.1. UX / utilisabilité

- pediatrics-2021-3-e21471.pdf

7.2. Indicateurs "métier" (non médicaux)

- Adhérence (fréquence d'usage), complétion du check-in émotion, progression mini-jeux, diversité des stratégies testées.
- Satisfaction parent/enfant, perception de compréhension émotionnelle (auto-échelle adaptée).

---

2. BENCHMARK

Nom de l'application	Objectif principal	Public cible	Fonctionnement
Emotion Faces	Reconnaissance des émotions faciales	Enfants TSA / TND	Images de visages, choix d'émotion
Autimo	Reconnaissance émotionnelle	Enfants TSA	Jeux basés sur photos réelles
Emotionary	Vocabulaire émotionnel	Enfants avec difficultés socio-émotionnelles	Dictionnaire émotionnel interactif
Mightier	Régulation émotionnelle	Enfants TSA / TND	Jeux
GoNoodle	Bien-être et gestion des émotions	Enfants (écoles)	Vidéos de respiration et mouvement
Smiling Mind	Méditation et bien-être	Enfants et adolescents	Exercices guidés
Avaz	Communication alternative	Enfants non verbaux / TSA	Pictogrammes personnalisables
Proloquo2Go	Communication augmentée	TSA / TND	Pictogrammes et synthèse vocale
Autism Tracker	Suivi comportemental	Parents d'enfants TSA	Entrées parentales / objet ajouté
Bearable	Suivi de l'humeur	Grand public	Tracking quotidien
Emoface	Reconnaissance des émotions faciales	Enfants TSA / TND	Analyse et identification d'émotions à partir de visages

Points communs avec COONY	Limites principales	Interessant pour le projet	Liens
Travail sur les émotions	Pas d'émotion vécue, pas de suivi parent		
Approche ludique	Pas de régulation émotionnelle		https://auticiel.com/application/autimo/
Lien émotion-mot	Peu personnalisé		https://play.google.com/store/apps/details?id=com.techpeak.emotionary
Apprentissage de stratégies	Matériel externe, coûteux	Jeux	https://www.mightier.com/fr-ca/
Ludique et accessible	Pas spécifique TSA		https://play.google.com/store/apps/details?id=com.gonoodle.gonoodle&hl=fr
Gestion du stress	Charge verbale élevée		https://www.smilingmind.com.au/
Support visuel	Peu centré émotions	Communication non verbale	https://www.avazapp.fr/
Expression des besoins	Émotions peu développées	Communication non verbale	https://www.assistiveware.com/fr/produits/proloquo2go
Suivi longitudinal	Enfant peu impliqué		
Analyse d'humeur	Non adapté aux enfants	Suivi journalier	https://bearable.app/
Travail sur les émotions, approche pédagogique	Émotions externes uniquement, pas de régulation ni de suivi parent	Manière d'aborder les émotions	https://www.emoface.fr/fr/

➕ BENCHMARK COONY

3. **MVP : Minimal Viable Product**

MVP COONY

Aider un enfant avec TSA à identifier et exprimer son état émotionnel, et fournir un premier niveau de soutien compréhensible et sécurisé, sans visée médicale.

PROBLÈME COEUR À RÉSOUDRE

Les enfants TSA présentent généralement des difficultés dans l'identification et la verbalisation des émotions. Aujourd'hui, peu d'outils de compréhension adaptés existent au quotidien et les parents manquent de visibilité simple sur l'état émotionnel de leur enfant.

Notre objectif : prouver que COONY est utilisable par un enfant TSA, facilite la mise en mots / images des émotions et permet au parent de se "connecter" aux émotions de l'enfant.

FONCTIONNALITÉ COEUR

1. **Check-in émotionnel guidé** (Indispensable)

- Sélection d'une émotion via pictogrammes / couleurs / formes
- Choix de l'intensité (ex : échelle 1–5)
- Option simple de ressenti corporel (ex : cœur qui bat fort, ventre serré)

Questionnaire à compléter quotidiennement permettant de rentrer les émotions ressenties par l'enfant dans la journée.

2. **Visualisation simple de l'émotion : Feedback immédiat à l'enfant**

Pour chaque question : émotion choisie / intensité / message rassurant.

Validation de l'émotion.

3. **Recommandation basique de stratégie : Soutien immédiat, non médical**

En fonction du résultat global du questionnaire, proposer un mini jeu adapté à la situation :

- Analyse globale des réponses au questionnaire émotionnel
- Proposition automatique d'un mini-jeu adapté via un système de règles simples.

Ex : Forte colère ressentie (colère + forte intensité) : mini jeu de respiration / apaisement

4. **Minis-jeux (min 1)**

Spécifique à l'émotion ressentie : un mini jeu = une réponse finale questionnaire

5. **Vue parent ultra simplifiée : Lecture uniquement**

Tableau de bord simplifié avec visualisation simple des émotions de l'enfant.

6. **Compte parent avec connexion**

Création et connexion au compte parent. Accès aux fonctionnalités à partir du compte parent.

FONCTIONNALITÉS ADDITIONNELLES

- **Personnalisation avancée**

Préférences sensorielles (sons, couleurs, animations)
Adaptation de l'interface à la sensibilité des enfants

- **Système de recommandation enrichi**

Adaptation selon l'historique,
Suggestion basée sur les préférences parent/enfant.

- **Bibliothèque de mini-jeux**

Plusieurs types de jeux adaptés à des analyses plus fines du questionnaire.
Progression forte.

- **Tableau de bord parent avancé**

Statistiques simples (fréquence, tendances).
Guide et suggestion d'aide pour les parents.

- **Notifications intelligentes**

Rappel quotidien pour questionnaire du jour.
Sélection des préférences

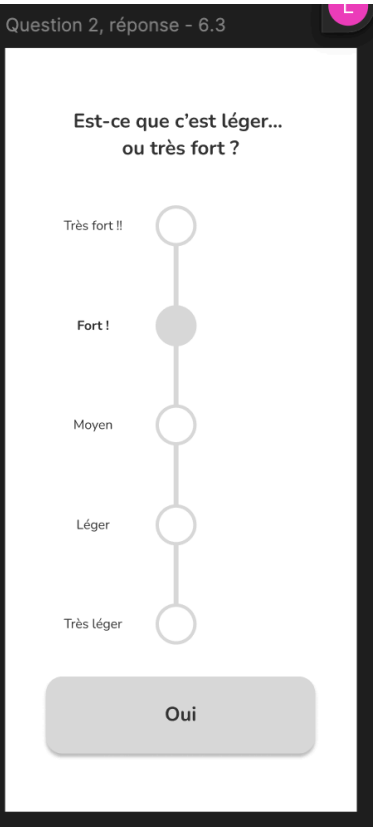
- **Multi-profils**

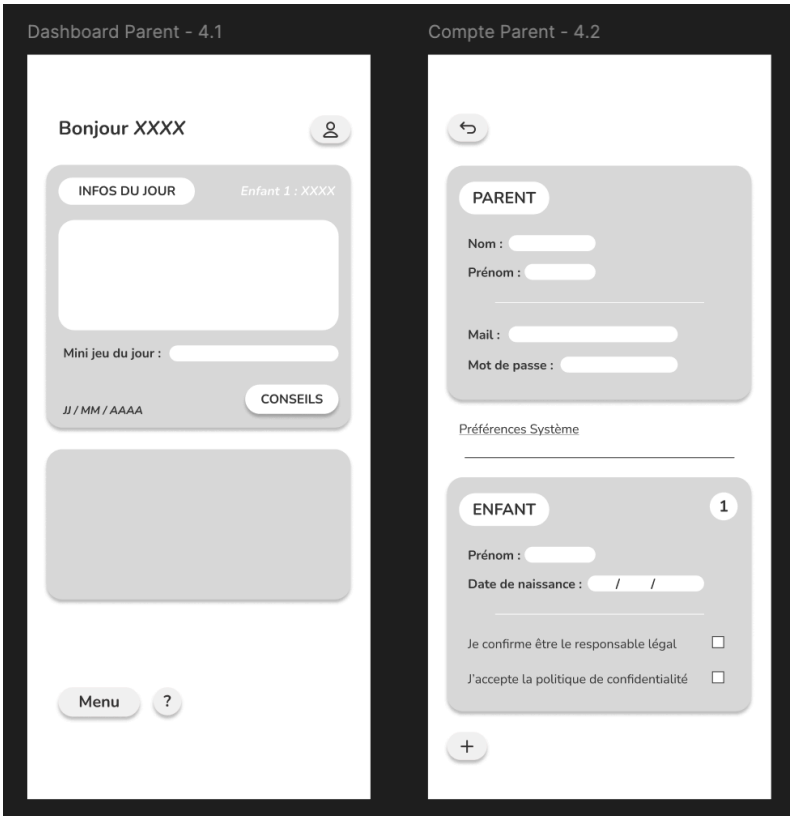
Possibilité que plusieurs parents se connectent au même compte. Et qu'il y est plusieurs enfants sur un même compte parent.

4. MAQUETTE FONCTIONNELLE FIGMA

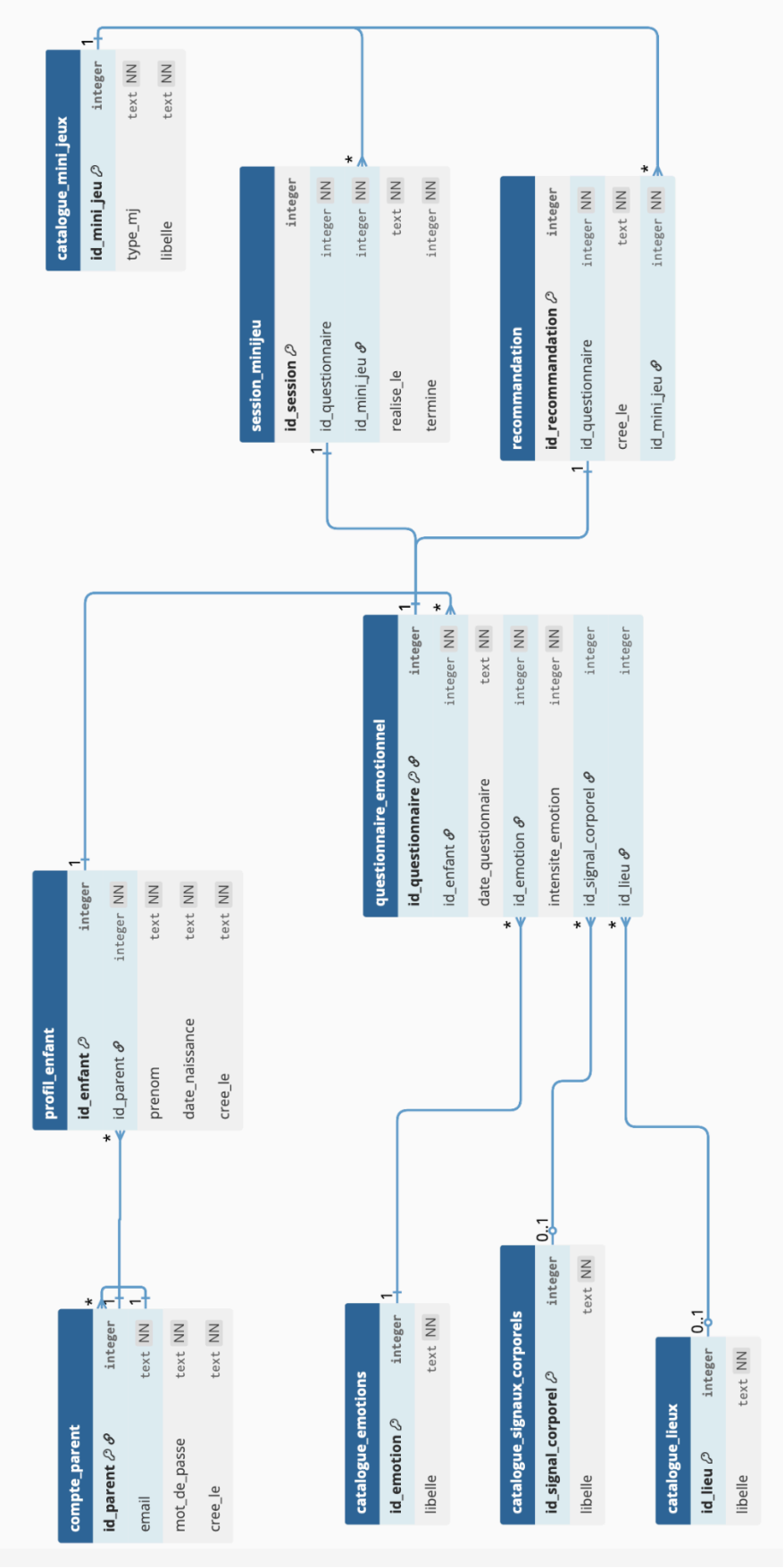
LIEN PROJET : Figma

LIEN PROTOTYPE :
https://www.figma.com/proto/LCCNdTuJoUJgGPICGaNEVl/Sans-titre?node-id=0-1&t=umuVhigJOx9dsRzv-1



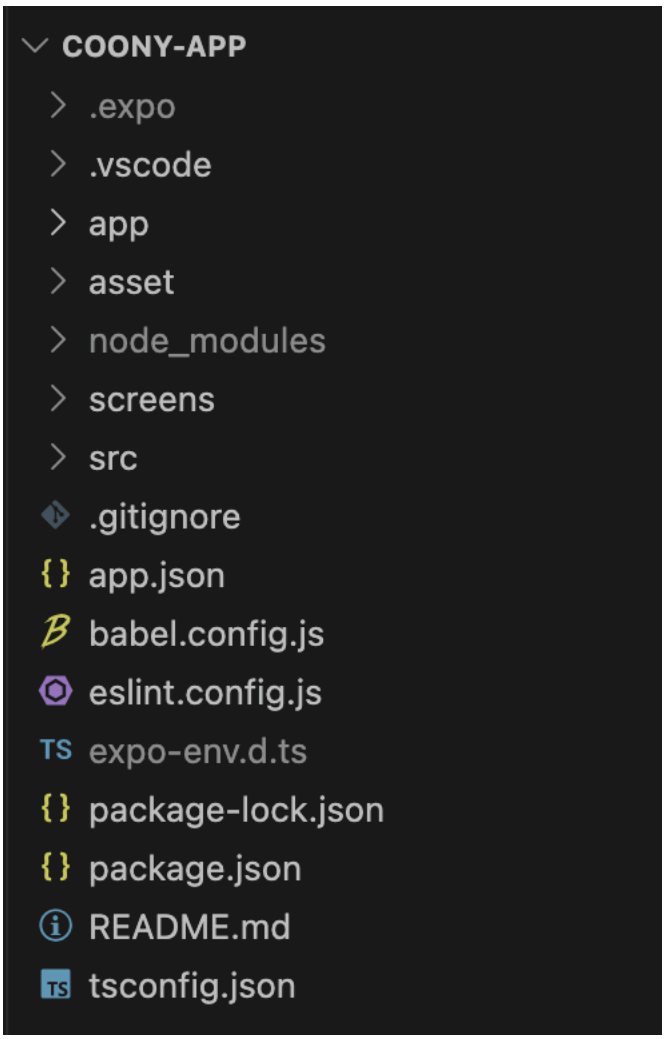


5. Schéma de la base de données



6. ARCHITECTURE DE L'APPLICATION

- Architecture globale



- Architecture détaillée

```
COONY-APP
  .expo
  .vscode
  app
    (auth)
      connexion.js
      index.js
      inscription.js
    (enfant)
      questionnaire
        corps.js
        emotion.js
        intensite.js
        intro.js
        lieu.js
        resultat.js
      tableau_de_bord_enfant.js
    (parent)
      ajout_enfant.js
      compte_parent.js
      menu.js
      tableau_de_bord_parent.js
    mini-jeu
      [id].js
    _layout.js
  asset/images
  node_modules
  screens
    auth
      PageAccueil.jsx
      PageConnexion.jsx
      PageInscription.jsx
    enfant
      questionnaire
        PageIntroQuestionnaire.jsx
        PageQuestionCorps.jsx
        PageQuestionEmotion.jsx
        PageQuestionIntensite.jsx
        PageQuestionLieu.jsx
        PageResultatQuestionnaire.jsx
      PageTableauBordEnfant.jsx
    mini-jeu
      PageMiniJeu.jsx
    parent
      PageAjoutEnfant.jsx
      PageCompteParent.jsx
      PageMenu.jsx
      PageTableauBordParent.jsx
```

```
src
  components
    common
      Button.jsx
      Card.jsx
      Input.jsx
      PageHeader.jsx
      Pastille.jsx
    dashboard
      ActivityCard.jsx
      EmotionSummary.jsx
    Mini_Jeux
      Mini_Jeu_Respiration.jsx
    parent
      ChildSelector.jsx
      HistoryItem.jsx
    EnteteQuestionnaire.jsx
    FondOnde.jsx
  data
    repositories
      enfant.repo.js
      parent.repo.js
      questionnaire.repo.js
      recommendation.repo.js
    seed
      catalogues.seed.js
      users.seed.js
    sqlite
      client.js
      queries.js
    mcd-mld-diagrams.pdf
    schema.js
  hooks
    auth
      useConnexion.js
      useInscription.js
    dashboard
      useEnfantTableauDeBord.js
      useParentTableauDeBord.js
    useAjoutEnfant.js
    useCompteParent.js
    useMenu.js
  services
    recommendation.service.js
  state
    enfantSelectionne.js
    questionnaireState.js
    sessionParent.js
```

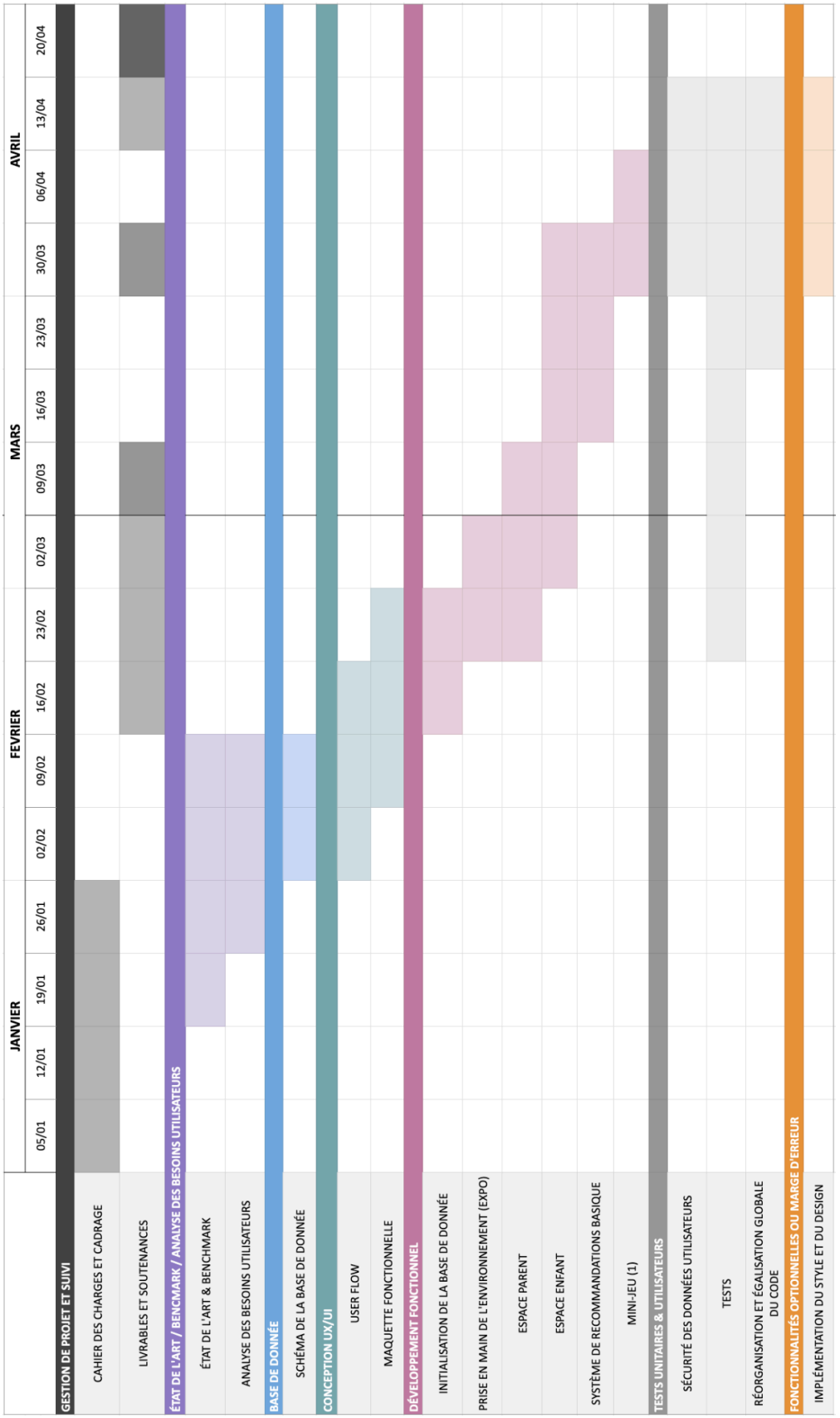
```
utils
  mapper
    emotionMapper.js
    miniJeuMapper.js
  colors.js
  date.js
  hash.js
  validator.js
.gitignore
app.json
babel.config.js
eslint.config.js
expo-env.d.ts
package-lock.json
package.json
README.md
tsconfig.json
```

- Architecture détaillée : *tree -I 'node_modules'*

```
.
├── README.md
├── app
│   ├── (auth)
│   │   ├── connexion.js
│   │   ├── index.js
│   │   └── inscription.js
│   ├── (enfant)
│   │   ├── questionnaire
│   │   │   ├── corps.js
│   │   │   ├── emotion.js
│   │   │   ├── intensite.js
│   │   │   ├── intro.js
│   │   │   ├── lieu.js
│   │   │   └── resultat.js
│   │   └── tableau_de_bord_enfant.js
│   ├── (parent)
│   │   ├── ajout_enfant.js
│   │   ├── compte_parent.js
│   │   ├── menu.js
│   │   └── tableau_de_bord_parent.js
│   ├── _layout.js
│   └── mini-jeu
│       └── [id].js
├── app.json
├── asset
│   └── images
├── babel.config.js
├── eslint.config.js
├── expo-env.d.ts
├── package-lock.json
├── package.json
├── screens
│   ├── auth
│   │   ├── PageAccueil.jsx
│   │   ├── PageConnexion.jsx
│   │   └── PageInscription.jsx
│   ├── enfant
│   │   ├── PageTableauBordEnfant.jsx
│   │   └── questionnaire
│   │       ├── PageIntroQuestionnaire.jsx
│   │       ├── PageQuestionCorps.jsx
│   │       ├── PageQuestionEmotion.jsx
│   │       ├── PageQuestionIntensite.jsx
│   │       ├── PageQuestionLieu.jsx
│   │       └── PageResultatQuestionnaire.jsx
│   ├── mini-jeu
│   │   └── PageMiniJeu.jsx
│   └── parent
│       ├── PageAjoutEnfant.jsx
│       ├── PageCompteParent.jsx
│       ├── PageMenu.jsx
│       └── PageTableauBordParent.jsx
```

```
├── src
│   ├── components
│   │   ├── EnteteQuestionnaire.jsx
│   │   ├── FondOnde.jsx
│   │   ├── Mini_Jeux
│   │   │   └── Mini_Jeu_Respiration.jsx
│   │   ├── common
│   │   │   ├── Button.jsx
│   │   │   ├── Card.jsx
│   │   │   ├── Input.jsx
│   │   │   ├── PageHeader.jsx
│   │   │   └── Pastille.jsx
│   │   ├── dashboard
│   │   │   ├── ActivityCard.jsx
│   │   │   └── EmotionSummary.jsx
│   │   └── parent
│   │       ├── ChildSelector.jsx
│   │       └── HistoryItem.jsx
│   ├── data
│   │   ├── mcd-mld-diagrams.pdf
│   │   ├── repositories
│   │   │   ├── enfant.repo.js
│   │   │   ├── parent.repo.js
│   │   │   ├── questionnaire.repo.js
│   │   │   └── recommendation.repo.js
│   │   ├── schema.js
│   │   ├── seed
│   │   │   ├── catalogues.seed.js
│   │   │   └── users.seed.js
│   │   └── sqlite
│   │       ├── client.js
│   │       └── queries.js
│   ├── hooks
│   │   ├── auth
│   │   │   ├── useConnexion.js
│   │   │   └── useInscription.js
│   │   ├── dashboard
│   │   │   ├── useEnfantTableauDeBord.js
│   │   │   └── useParentTableauDeBord.js
│   │   ├── useAjoutEnfant.js
│   │   ├── useCompteParent.js
│   │   └── useMenu.js
│   ├── services
│   │   └── recommendation.service.js
│   ├── state
│   │   ├── enfantSelectionne.js
│   │   ├── questionnaireState.js
│   │   └── sessionParent.js
│   └── utils
│       ├── colors.js
│       ├── date.js
│       ├── hash.js
│       ├── mapper
│       │   ├── emotionMapper.js
│       │   └── miniJeuMapper.js
│       └── validator.js
└── tsconfig.json
```

7. DIAGRAMME DE GANTT : PLANNING RÉEL



8. PROCESSUS D'INSTALLATION

TÉLÉCHARGER L'APPLICATION EXPO GO

Pour pouvoir utiliser COONY, il faut télécharger l'application Expo Go sur votre smartphone.

- **Apple** : https://apps.apple.com/fr/app/expo-go/id982107779

- **Android (Google Play Store)** https://play.google.com/store/apps/details?id=host.exp.exponent

INSTALLATION ET LANCEMENT DE L'APPLICATION

> Ouvrir un nouveau terminal et cloner le dépôt GIT

- git clone https://github.com/ton-utilisateur/coony-app.git

> Ouvrir le dossier cloné dans le terminal et entrer dans l'application

- cd coony-app

> Installer les dépendances et lancer le serveur de développement

- npm install
- npx expo start

OUVRIR L'APPLICATION COONY SUR SON SMARTPHONE

Le téléphone et l'ordinateur doivent être sur le même réseau wifi !

Une fois le code lancé, une message s'affiche dans le terminal :

> Scanner le QR CODE donné dans le terminal et suivez le lien vers l'application Expo go.

BIENVENUE SUR COONY !